

Sum of a sequence

$$\sum_{k=m}^n k = \frac{(m+n)(n-m+1)}{2} ; \sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$$
$$\sum_{k=m}^n z^k = \begin{cases} n-m+1 & z = 1 \\ \frac{z^m - z^{n+1}}{1-z} & z \neq 1 \end{cases}$$
$$\log_b n = \frac{\log_2 n}{\log_2 b} ; \log(ab) = \log a + \log b$$

1 Sorting

INSERTION-SORT(A, n)
 for j = 2 to n
 key = A[j]
 i = j - 1
 while i > 0 and A[i] > key
 A[i + 1] = A[i]
 i = i - 1
 A[i + 1] = key

Runtime & Loop invariant (like induction)

In-place. $\Theta(n^2)$ worst, $\Theta(n)$ best. **Inv:** $A[1..j-1]$ sorted. **Init:** $j=2$, trivial. **Maint:** key inserted correctly. **Term:** A sorted.

MERGE(A, p, q, r)
 n1 = q-p+1; n2 = r-q
 L[1..n1+1] and R[1..n2+1] be new arrays
 for i = 1 to n1: L[i] = A[p+i-1]
 for j = 1 to n2: R[j] = A[q+j]
 L[n1+1] = inf; R[n2+1] = inf
 i = 1; j = 1
 for k = p to r
 if L[i] <= R[j]: A[k] = L[i]; i = i+1
 else: A[k] = R[j]; j = j+1

MERGE-SORT(A, p, r)
 if p < r
 q = floor((p+r)/2)
 MERGE-SORT(A, p, q)
 MERGE-SORT(A, q+1, r)
 MERGE(A, p, q, r)

Runtime

Not in-place. $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$

2 Solving recurrences

The substitution method

1. Guess the form of the solution.
2. Use **induction** to find constants & verify.

Recursion trees

Draw tree, sum cost per level, sum all levels.

Master Theorem

$T(n) = aT(n/b) + f(n), a \geq 1, b > 1, \alpha = \log_b a:$
$$f = O(n^{\alpha-\varepsilon}) \Rightarrow T = \Theta(n^\alpha)$$
$$f = \Theta(n^\alpha) \Rightarrow T = \Theta(n^\alpha \log n)$$
$$f = \Omega(n^{\alpha+\varepsilon}), af(n/b) \leq cf(n) \Rightarrow T = \Theta(f(n))$$

3 Maximum-Subarray Problem

FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 ls=-inf; rs=-inf; sum=0
 for i=mid downto low // max: form A[i..mid]
 sum+=A[i]
 if sum>ls: ls=sum; ml=i
 sum=0
 for j=mid+1 to high // max: form A[mid+1..j]
 sum+=A[j]
 if sum>rs: rs=sum; mr=j
 return (ml, mr, ls+rs)

FIND-MAXIMUM-SUBARRAY(A, low, high)
 if high==low: return (low, high, A[low])
 mid=floor((low+high)/2)
 (l1,lh,ls) = FIND-MAXIMUM-SUBARRAY(A,low,mid)
 (r1,rh,rs) = FIND-MAXIMUM-SUBARRAY(A,mid+1,high)
 (c1,ch,cs) = FIND-MAX-CROSSING-SUBARRAY(A,low, mid,high)
 if ls>rs and ls>cs: return (l1,lh,ls)
 elseif rs>cs: return (r1,rh,rs)
 else: return (c1,ch,cs)

Runtime

$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \lg n)$

4 Matrix Multiplication

$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$. **Naive:** $\Theta(n^3)$ time, $\Theta(n^2)$ space.

Raw Divide-and-Conquer

Runtime

8 calls: $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$

Strassen's Algorithm using Divide-and-Conquer

Only **7** recursive calls. Partition as before, compute:

M1=(A11+A22)(B11+B22) M5=(A11+A12)B22
M2=(A21+A22)B11 M6=(A21-A11)(B11+B12)
M3=A11(B12-B22) M7=(A12-A22)(B21+B22)
M4=A22(B21-B11)
C11=M1+M4-M5+M7 C12=M3+M5
C21=M2+M4 C22=M1-M2+M3+M6

Runtime

$T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\lg 7}) \approx \Theta(n^{2.807})$

5 Karatsuba's Algorithm

Problem: multiply two n -digit integers x, y in base b .
Divide: split each number in two halves of $n/2$ digits:

$$x = x_H b^{n/2} + x_L \qquad y = y_H b^{n/2} + y_L$$
$$x \cdot y = x_H y_H \cdot b^n + (x_H y_L + x_L y_H) \cdot b^{n/2} + x_L y_L$$

Key: naively 4 products, but:

$$x_H y_L + x_L y_H = (x_H + x_L)(y_H + y_L) - x_H y_H - x_L y_L$$

So only **3** recursive multiplications needed.

Runtime

$T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$

6 Heaps

Assume $\text{LEFT}(i)$ and $\text{RIGHT}(i)$ are already max-heaps.

MAX-HEAPIFY(A, i, n)
 l = LEFT(i)
 r = RIGHT(i)
 if l <= n and A[l] > A[i]
 largest = l
 else largest = i
 if r <= n and A[r] > A[largest]
 largest = r
 if largest != i
 exchange A[i] with A[largest]
 MAX-HEAPIFY(A, largest, n)

Runtime

$T(n) \leq T(2n/3) + \Theta(1) = O(\lg n)$

BUILD-MAX-HEAP(A, n)
 for i = floor(n/2) downto 1
 MAX-HEAPIFY(A, i, n)

Runtime

$O(n)$ (tighter than the naive $O(n \lg n)$)

HEAPSORT(A, n)
 BUILD-MAX-HEAP(A, n)
 for i = n downto 2
 exchange A[1] with A[i]
 MAX-HEAPIFY(A, 1, i-1)

Runtime

$O(n \lg n)$, in-place.

Implementation of priority queue

HEAP-EXTRACT-MAX(A, n) # error if $n < 1$
 max, A[1] = A[1], A[n]
 MAX-HEAPIFY(A, 1, n-1)
 return max

HEAP-INCREASE-KEY(A, i, key) # error if $\text{key} < A[i]$
 A[i] = key
 while i > 1 and A[PARENT(i)] < A[i]
 exchange A[i] with A[PARENT(i)]
 i = PARENT(i)

MAX-HEAP-INSERT(A, key, n)
 n = n + 1
 A[n] = -inf
 HEAP-INCREASE-KEY(A, n, key)

Runtimes

Insert: $O(\lg n)$, Maximum: $O(1)$, Extract-Max: $O(\lg n)$, Increase-Key: $O(\lg n)$

7 Data Structures for Disjoint Sets

Each set identified by a representative.

Operations

- **MAKE-SET**(x): $S_i = \{x\}$, add to S
- **UNION**(x, y): $S = S \setminus \{S_x, S_y\} \cup \{S_x \cup S_y\}$
- **FIND**(x): return representative of x in S_x

CONNECTED-COMPONENTS(G)
 for each vertex v in G.V
 MAKE-SET(v)
 for each edge (u, v) in G.E
 if FIND(u) != FIND(v)
 UNION(u, v)

8 Traversing/Searching a Graph G=(V,E)

BFS(G, s)
 for each u in G.V - {s}
 u.color = WHITE; u.d = inf; u.pi = NIL
 s.color = GRAY; s.d = 0; s.pi = NIL
 Q = {s}
 while Q != {}
 u = DEQUEUE(Q)
 for each v in G.Adj[u]
 if v.color == WHITE
 v.color = GRAY
 v.d = u.d + 1; v.pi = u
 ENQUEUE(Q, v)
 u.color = BLACK

BFS properties

Search for all vertices from s. Runtime: $O(V + E)$.

DFS(G)

for each u in G.V: u.color = WHITE
 time = 0
 for each u in G.V
 if u.color == WHITE: DFS-VISIT(G, u)

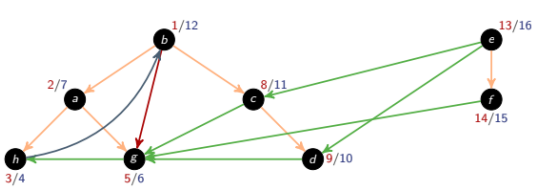
DFS-VISIT(G, u)
 time = time + 1; u.d = time
 u.color = GRAY
 for each v in G.Adj[u]
 if v.color == WHITE: DFS-VISIT(v)
 u.color = BLACK
 time = time + 1; u.f = time

Runtime

$\Theta(V + E)$

Classification of edges

Tree: found by exploring (u, v) in DF forest
Back: (u, v) , u descendant of v
Forward: (u, v) , v desc. of u , not tree edge
Cross: any other edge
UDG: only tree/back edges. | **DAG:** no back edge.



Parenthesis theorem
For all u, v , exactly one of: • Neither ances.: $u.d < u.f < v.d < v.f$ (or $>$) • v descendant of u: $u.d < v.d < v.f < u.f$ • u descendant of v: $v.d < u.d < u.f < v.f$

TOPOLOGICAL-SORT(G): Call DFS(G) to compute v.f for all v in G.V Output vertices in order of decreasing v.f

Runtime
Same as DFS: $\Theta(V + E)$

SCC(G): Call DFS(G) to compute u.f for all u Compute G^T (with all edges reversed) Call DFS(G^T), visiting vertices in decreasing u.f order Output each tree in DFS forest = one SCC
--

Runtime
Same as DFS: $\Theta(V + E)$

9 Flow network
Capacity constraint $\forall u, v \in V, 0 \leq f(u, v) \leq c(u, v)$

Cut
Partition of V into two disjoint sets S and T such that $s \in S$ and $t \in T$. $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$.

Maximum flow problem - Ford-Fulkerson method
Residual network $G_f = (V, E_f)$
$c_f(u, v)$: additional flow possible (forward edge) or removable (backward edge).

FORD-FULKERSON-METHOD(G, s, t) initialize flow f to 0 while exists an augmenting path p in G_f b = bottleneck(p) augment flow along p by b return f
--

Max-flow min-cut theorem ($\Leftrightarrow$)
• f is the maximum flow • G_f has no augmenting path • $f = c(S, T)$ for a minimum cut (S, T)

Applications of max flow
Bipartite matching as flow
$G = (V, E)$, $V = L \cup R$. Connect $s \rightarrow \forall u \in L$ and $\forall v \in R \rightarrow t$, all capacities 1. Run Ford-Fulkerson.

Edge-disjoint paths
Max # edge-disjoint paths from s to t = min edges to remove to disconnect s from t = max flow (c=1).

10 Spanning Trees
Cut property (safe edge)
Given any cut, a minimum-weight crossing edge belongs to some MST.

<pre> PRIM(G, w, r) // G undirected ; w: E -> R weights Q = {} // min-priority (sorted by w) for each u in G.V: u.key = inf u.pi = NIL // u.pi = parent in T INSERT(Q, u) DECREASE-KEY(Q, r, 0) // r.key = 0 while Q != {}: u = EXTRACT-MIN(Q) for each v in G.Adj[u]: if v in Q and w(u,v) < v.key: v.pi = u DECREASE-KEY(Q, v, w(u,v)) </pre>
--

Why does it work?
T is always a subtree of a MST.

Prim's runtime
$O(E \lg V)$: binary heap. $O(V \lg V + E)$: Fibo. heap.

KRUSKAL(G, w) A = {} for v in G.V: MAKE-SET(v) sort G.E by w non-decreasing for (u, v) in G.E (sorted): if FIND(u) != FIND(v) A += {(u,v)}; UNION(u, v) return A

Why does it work?
T is always a sub-forest of a MST.

Kruskal's runtime
$O(E \lg E) = O(E \lg V)$ (sorting dominates).

11 Shortest Paths
Problem variants
• Single-source: from s to all vertices • Single-dest.: reverse edges $\rightarrow$ single-source • Single-pair: no better worst case than single-s. • All-pairs: solve single-source for each vertex

Negative weights
Allowed but no negative-weight cycle.

Optimal substructure
Any subpath of a shortest path is itself a shortest path (proof by contradiction).

Helpful tools
• $d(v)$ > path cost from s to v (upper bound) • $\pi(v)$ is the predecessor of v in this shortest path

INIT-SINGLE-SOURCE(G, s) s.d = 0 for v in G.V: v.d = inf; v.pi = NIL RELAX(u, v, w) if v.d > u.d + w(u, v) v.d = u.d + w(u, v); v.pi = u

BELLMAN-FORD(G, w, s) INIT-SINGLE-SOURCE(G, s) for i = 1 to $ G.V - 1$ for (u, v) in G.E: RELAX(u, v, w)

Correctness
Invariant: $l(v)$ is at most the length of the shortest path from s to v using at most i edges after the i'th iteration. Intuition: Après l'itération i, pour tout sommet v dont le plus court chemin depuis s utilise au plus i arêtes, $l(v)$ vaut déjà sa valeur finale.

Negative cycle detection
Run a $ V $ -th iteration: if any edge (u, v) can still be relaxed, the shortest path to v would require more than $ V - 1$ edges, implying a cycle. Since relaxing it decreases the cost, that cycle must be negative.

Runtime
$\Theta(E \cdot V)$. Good for distributed routing: each vertex repeatedly asks neighbors for best path.

Non-negative weights only. (an algo like Prim's)
DIJKSTRA(G, w, s) INIT-SINGLE-SOURCE(G, s) S = {} Q = G.V // min-priority queue (sorted by d(v)) while Q != {}: u = EXTRACT-MIN(Q) S = S U {u} for each vertex v in G.Adj[u]: RELAX(u, v, w)

Loop invariant (correctness)
At the start of each iteration, we have for all $v \in S$ that the distance $v.d$ from s to v is equal to the shortest path from s to v .

Runtime
The runtime is the same as Prim's algorithm.

12 Probabilistic Analysis
Worst case rarely happens. Randomization helps avoid worst-case and adversarial inputs (e.g., random pivot in quicksort). Also necessary in cryptography.

Indicator RV
$I\{A\} = \begin{cases} 1 & A \text{ occurs} \\ 0 & \text{otherwise} \end{cases} \quad \mathbb{E}[I\{A\}] = P\{A\}$

Hiring Problem
Expected # of hires is $O(\ln n)$ vs worst case $O(n)$.

Birthday lemma
Prob. of <i>no</i> collision among q items drawn uniformly from $[n]$: $\leq e^{-q(q-1)/(2n)}$.

13 Data Structure: Hash Tables
Hash function
$h : U \rightarrow \{0, \dots, m-1\}$ maps key universe to m slots. Collision: $h(k_1) = h(k_2)$ with $k_1 \neq k_2$. Load factor: $\alpha = n/m$ (number of keys / slots).

Chaining (collision resolution)
Each slot: linked list of keys.

14 Quick Sort
Loop invariant of Partitioning
1. All entries in A[p...i] are $\leq$ pivot 2. All entries in A[i+1...j] are $>$ pivot 3. A[r] = pivot

PARTITION(A,p,r) x = A[r] i = p-1 for j = p to r - 1 if A[j] $\leq$ x i = i + 1 exchange A[i] with A[j] exchange A[i+1] with A[r] return i+1 QUICKSORT(A, p, r) if p < r q = PARTITION(A, p, r) QUICKSORT(A, p, q-1) QUICKSORT(A, q+1, r)
--

Runtime
In-place $\Theta(n^2)$ worst, $\Theta(n \lg(n))$ on best/average

RANDOMIZED-PARTITION(A, p, r) i = RANDOM(p, r) exchange A[r] with A[i] return PARTITION(A, p, r)

It changes how likely the worst case is to occur.